E4 Educator v2.0

T12

WiFi and MQTT

Tier 2 · Tutorial 5 of 9

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- How WiFi works on the ESP32 and the connection state machine
- How to store WiFi credentials securely in NVS rather than hardcoding
- How to handle WiFi reconnection robustly without blocking firmware
- What MQTT is, how it works, and why it suits embedded sensor networks
- How to set up the Mosquitto broker on MarkPi3
- How to publish sensor data and subscribe to command topics
- How to integrate WiFi and MQTT status into the TFT dashboard
- How to receive MQTT commands that control the E4 at runtime
- How to build a Node-RED dashboard that receives E4 sensor data

You will need:

- E4 Educator v2.0 board with ESP32 fitted
- T08, T11 completed — TFT pipeline and NVS settings established
- MarkPi3 Raspberry Pi with Mosquitto MQTT broker running
- Node-RED installed on MarkPi3 (optional — for dashboard)
- Button.h, Settings.h from previous tutorials

1 WiFi on the ESP32

The ESP32 includes an 802.11 b/g/n WiFi radio and Bluetooth 4.2 as built-in silicon — no external module required. WiFi is one of the ESP32's defining features and the reason it dominates IoT prototyping. Understanding how the connection process works prevents the most common WiFi programming mistakes.

1.1 The WiFi connection process

Connecting to WiFi is an asynchronous process with several distinct phases. Understanding each phase explains why blocking approaches fail and why a state machine is the right tool.

Phase	WiFi.status()	Description
Idle	`WL_IDLE_STATUS`	Radio initialised, not yet connecting
Connecting	`WL_DISCONNECTED`	Scanning for SSID, authenticating
Connected	`WL_CONNECTED`	IP address assigned, ready
No SSID	`WL_NO_SSID_AVAIL`	SSID not found in scan
Auth fail	`WL_CONNECT_FAILED`	Wrong password
Lost	`WL_CONNECTION_LOST`	Was connected, dropped

1.2 WiFi and ADC2

ADC2 reminder — WiFi active

When WiFi is connected, the WiFi radio uses ADC2 for internal calibration. Any analogRead() on ADC2 pins (GPIO 0, 2, 4, 12-15, 25-27) will return unreliable values. The E4 assigns all analog sensors to ADC1 pins (LDR=GPIO39, MIC=GPIO36) precisely because of this. Always use ADC1 pins in WiFi projects.

1.3 Storing credentials in NVS

Never hardcode WiFi credentials in your firmware. Hardcoded credentials are committed to version control, shared across boards, and impossible to change without reflashing. Store them in NVS instead.

```
// WiFi credentials in NVS — never in source code
#define NVS_NS_NET  "network"
#define NVS_SSID    "ssid"      // 4 chars ✓
#define NVS_PASS    "password"  // 8 chars ✓
#define NVS_BROKER  "broker"    // 6 chars ✓

String wifiSSID, wifiPass, mqttBroker;

void loadNetworkSettings() {
  Preferences p;
  p.begin(NVS_NS_NET, true);
  wifiSSID   = p.getString(NVS_SSID,   "");
  wifiPass   = p.getString(NVS_PASS,   "");
  mqttBroker = p.getString(NVS_BROKER, "192.168.1.79");
  p.end();
}
```

```
void saveNetworkSettings(const String& ssid,
                         const String& pass,
                         const String& broker) {
  Preferences p;
  p.begin(NVS_NS_NET, false);
  p.putString(NVS_SSID,   ssid);
  p.putString(NVS_PASS,   pass);
  p.putString(NVS_BROKER, broker);
  p.end();
  Serial.println("Network settings saved.");
}

// On first boot with empty NVS, fall back to compile-time defaults
// defined in a secrets.h file NOT committed to version control:
// #define DEFAULT_SSID   "lab_wifi"
// #define DEFAULT_PASS   "fundrum1"
// #define DEFAULT_BROKER "192.168.1.79"
```

2 Non-blocking WiFi connection

The most common WiFi mistake is using a blocking while loop to wait for connection. This freezes all firmware — display, buttons, sensors — for the entire connection duration which can be 5-30 seconds.

```
// ✗ BAD — blocking approach. Firmware freezes during connection.
WiFi.begin(ssid, pass);
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}
```

The correct approach uses a state machine and millis() — the same pattern used throughout Tier 1:

```
#include <WiFi.h>

enum WiFiState {
  WIFI_ST_IDLE,
  WIFI_ST_CONNECTING,
  WIFI_ST_CONNECTED,
  WIFI_ST_FAILED
};

WiFiState wifiState      = WIFI_ST_IDLE;
unsigned long wifiStart  = 0;
const unsigned long WIFI_TIMEOUT = 15000;  // 15 seconds

// Call from setup() to begin connection attempt
void wifiConnect() {
  if (wifiSSID.length() == 0) {
    Serial.println("No WiFi credentials — skipping.");
    wifiState = WIFI_ST_FAILED;
    return;
  }
  Serial.printf("Connecting to %s...\n", wifiSSID.c_str());
  WiFi.mode(WIFI_STA);
  WiFi.begin(wifiSSID.c_str(), wifiPass.c_str());
  wifiStart = millis();
  wifiState = WIFI_ST_CONNECTING;
}

// Call every loop() pass — non-blocking
void wifiUpdate(unsigned long now) {
  switch (wifiState) {
    case WIFI_ST_CONNECTING:
      if (WiFi.status() == WL_CONNECTED) {
        wifiState = WIFI_ST_CONNECTED;
        Serial.printf("WiFi connected. IP: %s\n",
                      WiFi.localIP().toString().c_str());
        digitalWrite(LED_GREEN, HIGH);
      } else if (now - wifiStart > WIFI_TIMEOUT) {
        wifiState = WIFI_ST_FAILED;
        Serial.println("WiFi timeout.");
```

```
        digitalWrite(LED_RED, HIGH);
      }
      break;

    case WIFI_ST_CONNECTED:
      // Monitor for drops
      if (WiFi.status() != WL_CONNECTED) {
        wifiState = WIFI_ST_CONNECTING;
        wifiStart = now;
        Serial.println("WiFi lost — reconnecting...");
        digitalWrite(LED_GREEN, LOW);
        WiFi.reconnect();
      }
      break;

    case WIFI_ST_FAILED:
      // Retry every 30 seconds
      if (now - wifiStart > 30000) {
        Serial.println("Retrying WiFi...");
        wifiConnect();
      }
      break;

    default: break;
  }
}

bool wifiReady() {
  return wifiState == WIFI_ST_CONNECTED;
}
```

Key point

The WiFi state machine follows the exact same pattern as the DS18B20 conversion state machine in T06 — IDLE, CONNECTING, CONNECTED, FAILED. The firmware continues to run, update the display, and respond to buttons throughout the entire connection process. The user sees progress on screen rather than a frozen device.

3 MQTT — the IoT messaging protocol

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol designed specifically for constrained devices and unreliable networks. It is the standard protocol for IoT sensor communication and is what connects the E4 to Node-RED and other home automation systems.

3.1 How MQTT works

MQTT uses a broker-client model. The broker (Mosquitto on MarkPi3) is a server that receives and routes messages. Clients (the E4, Node-RED, phone apps) connect to the broker and either publish messages to topics or subscribe to receive them.

Concept	Description
Broker	The message router. Mosquitto running on MarkPi3 at 192.168.1.79, port 1883.
Client	Any device that connects to the broker. E4, Node-RED, MQTT Explorer, phone apps.
Topic	A named message channel. Hierarchical with / separators. e.g. e4/sensor/temperature
Publish	Send a message to a topic. Any client can publish to any topic.
Subscribe	Register to receive messages on a topic. Broker delivers matching messages automatically.
QoS	Quality of Service level. 0 = fire and forget, 1 = at least once, 2 = exactly once. Use QoS 1 for sensor data.
Retain	Broker stores the last message on a topic. New subscribers immediately receive the last value.

3.2 E4 topic structure

Use a consistent hierarchical topic structure across all E4 projects:

Topic	Direction	Payload
`e4/sensor/temperature`	E4 → broker	Float as string: "21.4"
`e4/sensor/humidity`	E4 → broker	Float as string: "55.2"
`e4/sensor/light`	E4 → broker	Integer as string: "72"
`e4/status`	E4 → broker	"online" or "offline" (LWT)
`e4/cmd/brightness`	broker → E4	Integer 0-255: "200"
`e4/cmd/alert`	broker → E4	"on" or "off"

3.3 Mosquitto on MarkPi3

MarkPi3 runs Mosquitto as the MQTT broker at 192.168.1.79 on port 1883. To verify it is running:

```
# On MarkPi3 terminal:
sudo systemctl status mosquitto

# Test publish from MarkPi3:
mosquitto_pub -h localhost -t "test/hello" -m "world"
```

```
# Test subscribe from MarkPi3:
mosquitto_sub -h localhost -t "e4/#" -v
# The # wildcard subscribes to all e4/ subtopics

# From Windows PC on same network:
# Install MQTT Explorer (mqttexplorer.com)
# Connect to 192.168.1.79 port 1883
# Browse all topics visually
```

4 PubSubClient — MQTT library

PubSubClient is the standard MQTT library for Arduino/ESP32. It is lean, reliable, and widely used. Add it to platformio.ini:

```
lib_deps =
  bodmer/TFT_eSPI @ 2.5.43
  adafruit/Adafruit AHTX0
  adafruit/Adafruit BusIO
  knolleary/PubSubClient @ 2.8
```

4.1 MQTT connection and keep-alive

MQTT requires a persistent TCP connection to the broker. If the connection drops, you must reconnect and re-subscribe. PubSubClient handles the keep-alive ping automatically but you must call client.loop() every loop() pass to process incoming messages and maintain the connection.

```
#include <PubSubClient.h>
#include <WiFiClient.h>

WiFiClient   wifiClient;
PubSubClient mqtt(wifiClient);

#define MQTT_CLIENT_ID  "e4-educator"
#define MQTT_TOPIC_BASE "e4"

// Last Will and Testament — broker publishes this if E4 disconnects
#define MQTT_LWT_TOPIC   "e4/status"
#define MQTT_LWT_MSG     "offline"

// Received message callback
void mqttCallback(char* topic, byte* payload, unsigned int length) {
  String msg;
  for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];
  Serial.printf("MQTT rx [%s]: %s\n", topic, msg.c_str());

  // Handle commands
  if (String(topic) == "e4/cmd/brightness") {
    int b = msg.toInt();
    b = constrain(b, 0, 255);
    ledcWrite(0, b);
    Serial.printf("Brightness set to %d\n", b);
  }
  else if (String(topic) == "e4/cmd/alert") {
    if (msg == "on")  digitalWrite(LED_BLUE, HIGH);
    if (msg == "off") digitalWrite(LED_BLUE, LOW);
  }
}

bool mqttConnect() {
  if (!wifiReady()) return false;
```

```
  mqtt.setServer(mqttBroker.c_str(), 1883);
  mqtt.setCallback(mqttCallback);
  mqtt.setKeepAlive(60);

  Serial.printf("Connecting to MQTT broker %s...\n",
                mqttBroker.c_str());

  // Connect with Last Will and Testament
  bool ok = mqtt.connect(MQTT_CLIENT_ID,
                         nullptr, nullptr,        // no auth
                         MQTT_LWT_TOPIC, 1,       // LWT topic + QoS
                         true,                    // LWT retain
                         MQTT_LWT_MSG);           // LWT message
  if (ok) {
    mqtt.publish("e4/status", "online", true);   // Announce online
    mqtt.subscribe("e4/cmd/#");                  // Subscribe to commands
    Serial.println("MQTT connected.");
  } else {
    Serial.printf("MQTT failed. RC=%d\n", mqtt.state());
  }
  return ok;
}

// Call every loop() pass
void mqttUpdate(unsigned long now) {
  if (!wifiReady()) return;

  if (!mqtt.connected()) {
    static unsigned long lastRetry = 0;
    if (now - lastRetry > 5000) {
      lastRetry = now;
      mqttConnect();
    }
  }
  mqtt.loop();  // Process incoming messages and keep-alive
}
```

Key point

The Last Will and Testament (LWT) is a message the broker automatically publishes if the E4 disconnects unexpectedly — power loss, crash, or network failure. Setting e4/status to "offline" as the LWT and "online" on connect gives Node-RED and other subscribers reliable device presence information without any polling.

5 Publishing sensor data

Publishing is straightforward once WiFi and MQTT are connected. The pattern is: read sensor, format payload as string, publish to topic.

```
void publishSensorData(float tempC, float humidity, int lightPct) {
  if (!mqtt.connected()) return;

  char payload[16];

  // Temperature
  snprintf(payload, sizeof(payload), "%.1f", tempC);
  mqtt.publish("e4/sensor/temperature", payload, true);  // Retained

  // Humidity
  snprintf(payload, sizeof(payload), "%.1f", humidity);
  mqtt.publish("e4/sensor/humidity", payload, true);

  // Light
  snprintf(payload, sizeof(payload), "%d", lightPct);
  mqtt.publish("e4/sensor/light", payload, true);

  // JSON payload — all sensors in one message
  char json[80];
  snprintf(json, sizeof(json),
           "{\"temp\":%.1f,\"humidity\":%.1f,\"light\":%d}",
           tempC, humidity, lightPct);
  mqtt.publish("e4/sensor/all", json, false);

  Serial.printf("Published: T:%.1f H:%.1f L:%d\n",
                tempC, humidity, lightPct);
}

// Use snprintf not String for MQTT payloads.
// String class causes heap fragmentation in long-running firmware.
// char buffers with snprintf are the Fundrum standard.
```

Retained messages

Publishing with retain=true tells the broker to store the last message on that topic. When Node-RED or any new subscriber connects, it immediately receives the last retained value without waiting for the next publish cycle. Use retain=true for sensor values and device status. Use retain=false for commands and transient events.

6 Complete project — networked sensor dashboard

This project extends the T08 TFT dashboard with WiFi and MQTT connectivity. The display shows sensor values locally and simultaneously publishes them to the broker. MQTT commands control backlight brightness and trigger alerts remotely.

6.1 platformio.ini

```
[env:e4_educator_v2]
platform   = espressif32@6.6.0
board      = esp32dev
framework  = arduino
monitor_speed = 115200
monitor_port  = COM10
upload_port   = COM10

lib_deps =
  bodmer/TFT_eSPI @ 2.5.43
  adafruit/Adafruit AHTX0
  adafruit/Adafruit BusIO
  knolleary/PubSubClient @ 2.8

build_flags =
  -DFW_VERSION=\"1.0.0\"
  -DFW_DATE=\"2026-05-01\"
  -DLED_RED=16 -DLED_GREEN=26 -DLED_BLUE=32
  -DBTN_NEXT=13 -DBTN_ENT=14
  -DTFT_CS=5 -DTFT_DC=17 -DTFT_BL=12
  -DTFT_MOSI=23 -DTFT_SCLK=18 -DTFT_MISO=19
  -DTOUCH_CS=33 -DSD_CS=15
  -DI2C_SDA=21 -DI2C_SCL=22
  -DONE_WIRE=4 -DLDR=39 -DMIC_ANALOG=36
  -DMEMS_BCLK=32 -DMEMS_WS=35 -DMEMS_SD=34
  -DDAC_AUDIO=25 -DPIEZO=27
  -DUSER_SETUP_LOADED=1 -DILI9341_DRIVER=1
  -DTFT_WIDTH=320 -DTFT_HEIGHT=240
  -DTFT_MISO=19 -DTFT_MOSI=23 -DTFT_SCLK=18
  -DTFT_CS=5 -DTFT_DC=17 -DTFT_RST=-1
  -DLOAD_GLCD=1 -DLOAD_FONT2=1 -DLOAD_FONT4=1
  -DLOAD_GFXFF=1 -DSMOOTH_FONT=1
  -DSPI_FREQUENCY=27000000
  -DSPI_READ_FREQUENCY=20000000
  -DSPI_TOUCH_FREQUENCY=2500000
  ; WiFi/MQTT defaults (override via NVS provisioning)
  -DDEFAULT_SSID=\"lab_wifi\"
  -DDEFAULT_PASS=\"fundrum1\"
  -DDEFAULT_BROKER=\"192.168.1.79\"
```

6.2 main.cpp

```
#include <Arduino.h>
#include <Wire.h>
```

```
#include <WiFi.h>
#include <WiFiClient.h>
#include <PubSubClient.h>
#include <Preferences.h>
#include <TFT_eSPI.h>
#include <Adafruit_AHTX0.h>
#include "Button.h"

// ── Hardware ──────────────────────────────────────────────
TFT_eSPI       tft;
TFT_eSprite    spr = TFT_eSprite(&tft);
Adafruit_AHTX0 aht;
Button         nextBtn(BTN_NEXT);
Button         entBtn(BTN_ENT);
WiFiClient     wifiClient;
PubSubClient   mqtt(wifiClient);

// ── Colour palette ────────────────────────────────────────
#define COL_BG      0x0000
#define COL_HEADER  0x0319
#define COL_FOOTER  0x18C3
#define COL_TEXT    0xFFFF
#define COL_LABEL   0xC618
#define COL_OK      0x07E0
#define COL_WARN    0xFFE0
#define COL_ERR     0xF800

// ── Zone layout ───────────────────────────────────────────
#define ZONE_HEADER_Y   0
#define ZONE_HEADER_H  40
#define ZONE_CONTENT_Y 40
#define ZONE_CONTENT_H 165
#define ZONE_FOOTER_Y  280
#define ZONE_FOOTER_H  40
#define SCR_W         320

// ── Network settings ──────────────────────────────────────
#define NVS_NS_NET  "network"
#define NVS_SSID    "ssid"
#define NVS_PASS    "password"
#define NVS_BROKER  "broker"

String wifiSSID, wifiPass, mqttBroker;

// ── State ─────────────────────────────────────────────────
enum WiFiSt { WIFI_IDLE, WIFI_CONNECTING, WIFI_CONNECTED, WIFI_FAILED };
WiFiSt wifiSt        = WIFI_IDLE;
unsigned long wifiT  = 0;

float tempC = 0, humidity = 0;
int   lightPct = 0;
unsigned long lastSensor = 0;
unsigned long lastFooter = 0;
const unsigned long SENSOR_INTERVAL = 5000;
```

```
const unsigned long FOOTER_INTERVAL = 1000;

// ── Network settings load ──────────────────────────────
void loadNetSettings() {
  Preferences p;
  p.begin(NVS_NS_NET, true);
  wifiSSID   = p.getString(NVS_SSID,   DEFAULT_SSID);
  wifiPass   = p.getString(NVS_PASS,   DEFAULT_PASS);
  mqttBroker = p.getString(NVS_BROKER, DEFAULT_BROKER);
  p.end();
}

// ── WiFi state machine ─────────────────────────────────
void wifiConnect() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(wifiSSID.c_str(), wifiPass.c_str());
  wifiT = millis();
  wifiSt = WIFI_CONNECTING;
  Serial.printf("Connecting to %s\n", wifiSSID.c_str());
}

bool wifiReady() { return wifiSt == WIFI_CONNECTED; }

void wifiUpdate(unsigned long now) {
  switch (wifiSt) {
    case WIFI_CONNECTING:
      if (WiFi.status() == WL_CONNECTED) {
        wifiSt = WIFI_CONNECTED;
        Serial.printf("WiFi OK. IP: %s\n",
                      WiFi.localIP().toString().c_str());
      } else if (now - wifiT > 15000) {
        wifiSt = WIFI_FAILED; wifiT = now;
        Serial.println("WiFi timeout.");
      }
      break;
    case WIFI_CONNECTED:
      if (WiFi.status() != WL_CONNECTED) {
        wifiSt = WIFI_CONNECTING; wifiT = now;
        WiFi.reconnect();
      }
      break;
    case WIFI_FAILED:
      if (now - wifiT > 30000) wifiConnect();
      break;
    default: break;
  }
}

// ── MQTT ──────────────────────────────────────────────
void mqttCallback(char* topic, byte* payload, unsigned int len) {
  String msg;
  for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];
  Serial.printf("MQTT [%s]: %s\n", topic, msg.c_str());
```

```
  if (String(topic) == "e4/cmd/brightness")
    ledcWrite(0, constrain(msg.toInt(), 0, 255));
  else if (String(topic) == "e4/cmd/alert") {
    digitalWrite(LED_BLUE, msg == "on" ? HIGH : LOW);
  }
}

bool mqttConnect() {
  if (!wifiReady()) return false;
  mqtt.setServer(mqttBroker.c_str(), 1883);
  mqtt.setCallback(mqttCallback);
  bool ok = mqtt.connect("e4-educator", nullptr, nullptr,
                         "e4/status", 1, true, "offline");
  if (ok) {
    mqtt.publish("e4/status", "online", true);
    mqtt.subscribe("e4/cmd/#");
    Serial.println("MQTT connected.");
  }
  return ok;
}

void mqttUpdate(unsigned long now) {
  if (!wifiReady()) return;
  if (!mqtt.connected()) {
    static unsigned long lastRetry = 0;
    if (now - lastRetry > 5000) { lastRetry = now; mqttConnect(); }
  }
  mqtt.loop();
}

void publishSensors() {
  if (!mqtt.connected()) return;
  char buf[16];
  snprintf(buf, sizeof(buf), "%.1f", tempC);
  mqtt.publish("e4/sensor/temperature", buf, true);
  snprintf(buf, sizeof(buf), "%.1f", humidity);
  mqtt.publish("e4/sensor/humidity", buf, true);
  snprintf(buf, sizeof(buf), "%d", lightPct);
  mqtt.publish("e4/sensor/light", buf, true);
}

// ── LDR ──────────────────────────────────────────────────
int readLDR() {
  long t = 0;
  for (int i = 0; i < 8; i++) { t += analogRead(LDR); delay(2); }
  return constrain(map(t/8, 200, 3800, 0, 100), 0, 100);
}

// ── Display ──────────────────────────────────────────────
void drawChrome() {
  tft.fillScreen(COL_BG);
  tft.fillRect(0, ZONE_HEADER_Y, SCR_W, ZONE_HEADER_H, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
```

```
  tft.setCursor(8, 12);
  tft.print("E4 Net Dashboard");
  tft.drawFastHLine(20, 160, 200, COL_LABEL);
  tft.setTextColor(COL_LABEL, COL_BG);
  tft.setTextSize(2);
  tft.setCursor(8, 48);  tft.print("TEMPERATURE");
  tft.setCursor(8, 172); tft.print("HUMIDITY");
}

void updateContent() {
  // Temperature
  uint16_t tc = (tempC > 28.0) ? COL_ERR :
                (tempC > 24.0) ? COL_WARN : COL_OK;
  spr.fillSprite(COL_BG);
  spr.setTextColor(tc, COL_BG);
  spr.setTextSize(4);
  spr.setCursor(8, 10);
  spr.printf("%.1f C", tempC);
  spr.setTextColor(COL_LABEL, COL_BG);
  spr.setTextSize(1);
  spr.setCursor(8, 58);
  spr.printf("Light: %d%%", lightPct);
  spr.pushSprite(0, 70);

  // Humidity
  uint16_t hc = (humidity > 70.0) ? COL_ERR :
                (humidity > 60.0) ? COL_WARN : COL_OK;
  spr.fillSprite(COL_BG);
  spr.setTextColor(hc, COL_BG);
  spr.setTextSize(4);
  spr.setCursor(8, 10);
  spr.printf("%.1f %%", humidity);
  spr.pushSprite(0, 192);
}

void updateFooter(unsigned long now) {
  const char* wSt = wifiReady()  ? "WiFi OK" :
                    (wifiSt == WIFI_CONNECTING) ? "WiFi..." : "No WiFi";
  const char* mSt = mqtt.connected() ? "MQTT OK" : "MQTT...";
  uint16_t wCol = wifiReady()     ? COL_OK : COL_WARN;
  uint16_t mCol = mqtt.connected() ? COL_OK : COL_WARN;

  spr.fillSprite(COL_FOOTER);
  spr.setTextSize(1);
  spr.setTextColor(wCol, COL_FOOTER);
  spr.setCursor(4,  12); spr.print(wSt);
  spr.setTextColor(mCol, COL_FOOTER);
  spr.setCursor(80, 12); spr.print(mSt);
  spr.setTextColor(COL_LABEL, COL_FOOTER);
  spr.setCursor(156, 12);
  spr.printf("Up:%lus", now / 1000);
  spr.pushSprite(0, ZONE_FOOTER_Y);
}
```

```
// ── Setup ─────────────────────────────────────────────────────
void setup() {
  Serial.begin(115200);
  Serial.printf("T12 Net Dashboard  FW:%s\n", FW_VERSION);

  Wire.begin(I2C_SDA, I2C_SCL);
  pinMode(LED_GREEN, OUTPUT); digitalWrite(LED_GREEN, LOW);
  pinMode(LED_RED,   OUTPUT); digitalWrite(LED_RED,   LOW);
  pinMode(LED_BLUE,  OUTPUT); digitalWrite(LED_BLUE,  LOW);
  nextBtn.begin();
  entBtn.begin();

  pinMode(TFT_BL, OUTPUT); digitalWrite(TFT_BL, LOW);
  tft.init();
  tft.setRotation(0);   // Landscape — correct for E4 Educator v2.0
  tft.fillScreen(COL_BG);
  ledcSetup(0, 5000, 8);
  ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, 200);

  spr.setColorDepth(16);
  spr.createSprite(SCR_W, 80);

  if (!aht.begin(&Wire)) {
    Serial.println("AHT20 not found.");
    digitalWrite(LED_RED, HIGH);
  }

  loadNetSettings();
  drawChrome();
  wifiConnect();
  Serial.println("Ready.");
}

// ── Loop ──────────────────────────────────────────────────────
void loop() {
  unsigned long now = millis();
  Button::Event nextEv = nextBtn.read();

  wifiUpdate(now);
  mqttUpdate(now);

  bool shouldRead = (nextEv == Button::SHORT_PRESS) ||
                    (now - lastSensor >= SENSOR_INTERVAL);

  if (shouldRead) {
    lastSensor = now;
    sensors_event_t h, t;
    aht.getEvent(&h, &t);
    tempC    = t.temperature;
    humidity = h.relative_humidity;
    lightPct = readLDR();
    updateContent();
    publishSensors();
```

```
  }

  if (now - lastFooter >= FOOTER_INTERVAL) {
    lastFooter = now;
    updateFooter(now);
  }
}
```

7 Node-RED dashboard

Node-RED running on MarkPi3 can subscribe to the E4's MQTT topics and display sensor data in a browser-based dashboard. This section covers the minimum flow to get data from the E4 onto a Node-RED gauge.

7.1 Install Node-RED dashboard

```
# On MarkPi3 — install dashboard nodes
cd ~/.node-red
npm install node-red-dashboard
sudo systemctl restart nodered

# Access Node-RED: http://192.168.1.79:1880
# Dashboard:       http://192.168.1.79:1880/ui
```

7.2 Minimum flow for E4 sensor display

In Node-RED, create this flow by dragging nodes from the palette:

Node type	Configuration	Purpose
mqtt in	Topic: e4/sensor/temperature Server: 192.168.1.79:1883	Receives temperature from E4
gauge	Min: 0, Max: 40, Unit: °C Label: Temperature	Displays temperature gauge on dashboard
mqtt in	Topic: e4/sensor/humidity Server: 192.168.1.79:1883	Receives humidity from E4
gauge	Min: 0, Max: 100, Unit: % Label: Humidity	Displays humidity gauge on dashboard
mqtt out	Topic: e4/cmd/brightness Server: 192.168.1.79:1883	Sends brightness command to E4
slider	Min: 0, Max: 255, Step: 10 Label: E4 Brightness	Dashboard slider that publishes to brightness topic

Connect each mqtt in node to its corresponding gauge node. Connect the slider node to the mqtt out node. Click Deploy. Open the dashboard at http://192.168.1.79:1880/ui and watch the E4 sensor data appear in real time.

Try this

Move the brightness slider in Node-RED and watch the E4 TFT backlight change in real time. This is the E4 receiving an MQTT command from Node-RED via MarkPi3 — a complete bidirectional IoT connection. The E4 is now a proper networked device.

8 T12 summary

Concept	Key takeaway
WiFi state machine	IDLE → CONNECTING → CONNECTED → FAILED. Non-blocking via millis(). Automatic reconnection on drop. 15s timeout, 30s retry.

Credentials in NVS	Never hardcode WiFi credentials. Store SSID, password, broker IP in NVS "network" namespace. Fall through to build_flag defaults on first boot.
ADC2 + WiFi	WiFi active disables ADC2. E4 uses ADC1 pins for all sensors. Never use GPIO 0,2,4,12-15,25-27 for analog when WiFi is enabled.
MQTT broker	Mosquitto on MarkPi3 at 192.168.1.79:1883. Topic hierarchy: e4/sensor/xxx for data, e4/cmd/xxx for commands.
PubSubClient	mqtt.loop() every loop() pass is mandatory. Without it, incoming messages are not processed and keep-alive fails.
LWT	Last Will and Testament: broker publishes "offline" to e4/status if E4 disconnects unexpectedly. Publish "online" on connect.
Retained messages	retain=true: broker stores last value, new subscribers receive it immediately. Use for sensor values and status. Not for commands.
snprintf payloads	Always use char buffers with snprintf() for MQTT payloads. The String class causes heap fragmentation in long-running firmware.
Footer status	Show WiFi and MQTT connection status in the TFT footer zone with colour coding. Green = connected, yellow = connecting/failed.

What's next

- T13 — LittleFS: now that WiFi is working, storing files in the ESP32's own flash (LittleFS) enables serving web pages, loading smooth fonts, and storing config without an SD card. LittleFS complements NVS for larger internal file storage.
- T14 — OTA updates: with WiFi established, updating firmware over-the-air without a USB cable is the natural next capability. ElegantOTA makes this straightforward and it is an essential feature for any deployed device.